

Relatório: Solucionador Tipado de Cubo Mágico 2x2

Disciplina: Desenvolvimento Guiado por Tipos (Haskell)

Projeto: `haskell-cube-solver`

Autor: Samuel Oliveira Costa

RA: 11202510049

Link Youtube: <https://youtu.be/a4dcrlTR0Lk>

1. O Projeto

O `haskell-cube-solver` é um motor formal e solucionador (*solver*) para o **Cubo Mágico 2x2x2 (Pocket Cube)** desenvolvido em Haskell, aplicando os conceitos de **Desenvolvimento Guiado por Tipos**.

O objetivo principal foi modelar regras físicas e matemáticas do cubo diretamente no sistema de tipos do GHC:

- **DataKinds e TypeFamilies:** Promoção das 6 cores para o nível de tipos e mapeamento de faces opostas (*Opuesto*).
- **GADTs e Singletons:** Construção segura de quinas válidas (*Quina c1 c2 c3*) via *smart constructors*, impedindo peças com cores repetidas ou opostas.
- **Tipos Existenciais** (*SomeCor*, *SomeQuina*, *SomeCubo*): Ponte segura entre a leitura de arquivos de texto em tempo de execução (*IO*) e o motor estritamente tipado.
- **Phantom Types:** Rastreamento do estado estrutural do cubo (*Cubo 'Embaralhado* VS. *Cubo 'Resolvido*).
- **Validação Algébrica:** Checagem de conjunto canônico, quiralidade 3D (peças não espelhadas) e paridade de *twist* ($\sum \text{twist} \equiv 0 \pmod{3}$).
- **Solver BFS Bidirecional:** Algoritmo que encontra a menor sequência de rotações (*U*, *R*, *F* e inversos) expandindo simultaneamente a partir do estado inicial e do estado resolvido alvo.

2. Como Utilizar

O gerenciamento de compilação, execução e testes é feito via **Stack**:

```
# Caso ainda não tenha sido compilado
stack build

# Resolve um cubo específico (passando o arquivo como argumento)
stack run -- cubos/cubo_3.txt # (Nota: executar apenas 'stack run' tentará carregar o arquivo 'cubo.txt' na raiz)

# Rodar a bateria de testes automatizados
stack test

# Visualização gráfica 3D (script auxiliar em Python/Matplotlib)
pip install -r requirements.txt
python cube_viewer.py cubos/cubo_1.txt
```

3. Destaques, Dificuldades e Surpresas

Destaques

- **Garantia estrutural por tipos:** O uso de GADTs e *smart constructors* garantiu que quinas inválidas (com cores repetidas ou faces opostas) não possam ser criadas, enquanto a validação de quiralidade 3D barrou peças espelhadas que não existem no mundo real.
- **Desempenho da BFS Bidirecional:** Fazer a busca partindo ao mesmo tempo do estado inicial e do estado alvo reduziu bastante o espaço de busca, encontrando a solução ótima em fração de segundo mesmo para cubos bem embaralhados.

Dificuldades

- **Geometria 3D e rotação das peças:** A parte mais trabalhosa foi modelar as rotações das faces (*U*, *R*, *F*) mantendo a orientação relativa das cores de cada quina correta ao trocar as peças de posição entre os 8 slots fixos.
- **Casamento de tipos existenciais com a lógica pura:** Como a leitura do arquivo devolve tipos empacotados (*SomeQuina*), foi preciso estruturar bem os desempacotamentos e *pattern matchings* para reaproveitar funções puras sem deixar o código confuso.

Surpresas

- **Impacto das validações prévias:** Implementar a checagem de paridade de *twist* ($\sum \text{twist} \equiv 0 \pmod{3}$) e quiralidade antes do solver evitou que o programa ficasse preso procurando solução para cubos fisicamente impossíveis.
- **Praticidade dos tipos existenciais:** No início parecia complicado transitar do arquivo TXT para o sistema de tipos estrito, mas os tipos existenciais se mostraram uma forma muito direta e segura de isolar a entrada de dados.